

Лекция 4

Память агента

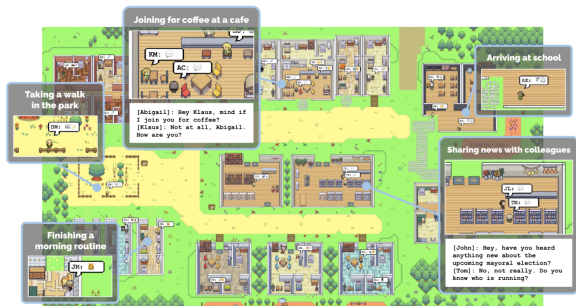
контекстное окно как бюджет, суммаризация, эпизодическая и семантическая
память

- ❶ **Проблема:** у модели нет памяти, а история дорожает
- ❷ **Контекст как бюджет:** почему нельзя просто тащить всё
- ❸ **Короткая память:** окно и суммаризация
- ❹ **Долгая память:** эпизодическая и семантическая, забывание

Связь с прошлой неделей

На первой неделе память агента была просто списком сообщений. Теперь этот список вырос, и им пора управлять: что оставить, что сжать, что достать заново.

Что бывает, когда у агентов есть память



Источник: Park et al., 2023. Generative Agents

- Стэнфордский эксперимент: 25 агентов живут в игровом городке, у каждого журнал всего, что он видел и делал
- Из журнала агент достаёт нужное, делает выводы и строит план на день
- Одному велели устроить вечеринку. За два игровых дня о ней узнала дюжина агентов, часть пришла вовремя
- Модель одна, без дообучения. Разница только в том, **что кладут в контекст**

- Модель это чистая функция: что прислали на вход, из того и родился ответ
- Между двумя вызовами она забывает всё. Помнит не модель, а **ваш код**, который заново шлёт историю
- Значит, «память агента» это инженерное решение: что именно класть во входной список на каждом шаге
- Хорошая новость: памятью можно управлять. Плохая: за неё платят каждый вызов

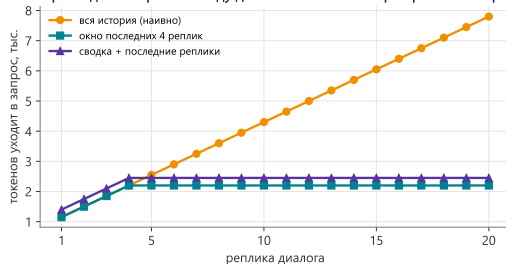
Два горизонта памяти

Короткая живёт внутри одной задачи: история шагов текущего диалога.

Долгая переживает перезапуск: что агент знает о пользователе и прошлых разговорах.

Наивная память: тащим всю историю

Размер каждого запроса по ходу диалога: наивная история растёт без предела



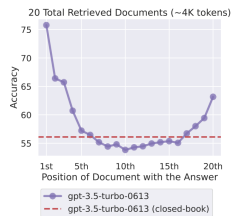
- Самый простой агент кладёт в запрос всю историю целиком. Работает, пока диалог короткий
- Каждая реплика добавляет токенов, и запрос пухнет линейно, а платим мы за каждый запрос
- Окно и суммаризация держат размер запроса плоским. Разница видна уже к десятой реплике

- Окно в миллион токенов соблазняет положить туда всё. Не поддавайтесь
- **Деньги**: каждый вызов оплачивает весь контекст. Длинная история на десятом шаге стоит как небольшая книга
- **Скорость**: чем длиннее вход, тем дольше ответ
- **Качество**: в длинном контексте модель хуже находит нужное. Это отдельный слайд

Правило бюджета

В контекст кладём не всё, что есть, а **минимум, которого хватит для следующего шага**. Всё остальное держим снаружи и достаём по необходимости.

Даже если влезает, теряется: потерян в середине



Наш опыт: один факт среди 240 похожих строк, меняем только его позицию

Оригинальная статья: та же U-образная кривая на 20 документах

Источник: Liu et al., 2023. Lost in the Middle

- В начале и в конце контекста факт находится почти всегда, **в середине проваливается**. И так у разных моделей
- Вывод не «модель плохая», а «длинный контекст ненадёжен». Важное нельзя топить в середине истории

Три способа не тащить всё

Обрезать

оставить последние реплики

дёшево, теряет старое

Сжать

свернуть старое в сводку

помнит суть, стоит вызова

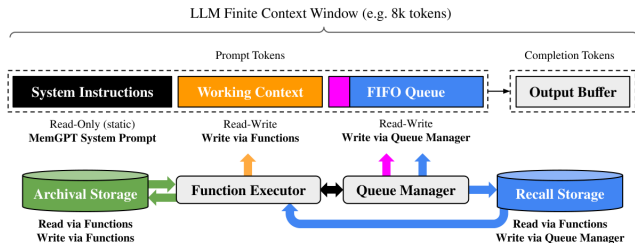
Достать заново

искать нужное по запросу

это RAG, следующие лекции

- Первые два это **короткая память**: управляем историей текущего диалога
- Третий это уже **долгая память** и знания: их разберём на пятой и шестой лекциях, сегодня подготовим почву
- В реальном агенте работают все три сразу, каждый на своём слое

Та же мысль в статье: контекст как оперативная память



Источник: Packer et al., 2023. MemGPT: Towards LLMs as Operating Systems

- MemGPT сравнивает контекст с оперативной памятью, а внешние хранилища с диском. Модель сама «подкачивает» нужное вызовами функций
- В окне живут три вещи: неизменные инструкции, рабочий контекст с фактами и очередь последних реплик. Всё остальное снаружи
- Это ровно наши три способа, собранные в одну систему. Сегодня идея живёт в проекте Letta

- Держим в контексте системный промпт и последние k сообщений, старое отбрасываем
- Плюс: размер запроса постоянный, цена не растёт
- Минус: агент забывает начало разговора. Спросили имя в первой реплике, к двадцатой его уже нет
- Годится, когда важен только недавний контекст: пошаговое решение задачи, короткий диалог

Что нельзя терять

Системный промпт и текущую задачу держим **всегда**, они не часть скользящего окна. Режем только промежуточные реплики, а не инструкции.

Суммаризация: свернуть старое в сводку

```
if tokens(history) > LIMIT:
    old = history[:-KEEP]          # что сворачиваем
    summary = model(
        "Сожми диалог в 5 фактов, "
        "сохрани имена, числа, решения:\n"
        + render(old))
    history = [summary] + history[-KEEP:]
```

Старое заменяем сводкой, последние реплики оставляем как есть.

- Помнит суть длинного разговора при коротком запросе
- Стоит **лишнего вызова** модели, и сводка может потерять деталь
- Запускаем не каждый шаг, а по порогу: перевалило за лимит, свернули
- Что беречь в сводке: имена, числа, принятые решения, открытые вопросы

Долгая память: два разных вида

Эпизодическая

что произошло:
лог прошлых диалогов,
действия, результаты

«вчера обсуждали доставку в Казань»

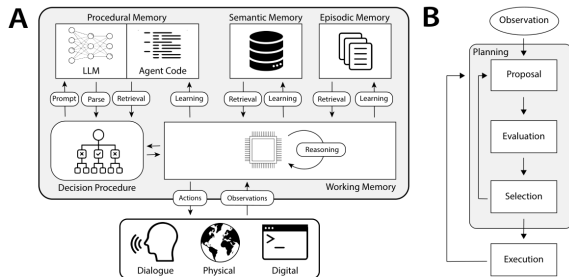
Семантическая

что известно:
факты о пользователе
и мире, без привязки к дате

«пользователя зовут Макар, город Казань»

- Различие как у людей: помнить *событие* и помнить *факт*. Это разные хранилища и разное применение
- Эпизоды растут бесконечно, поэтому из них выжимают факты. Факты компактны и попадают прямо в промпт

Четыре памяти по науке: CoALA

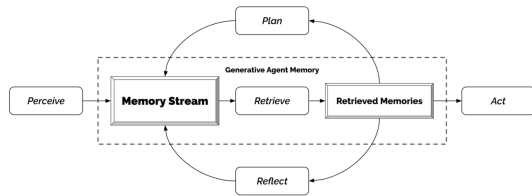


Источник: Sumers et al., 2023. Cognitive Architectures for Language Agents

- **Рабочая** память это сам контекст текущего шага
- **Эпизодическая** и **семантическая** те же, что у нас: события и факты
- **Процедурная** память это код агента и его промпты, то есть умение действовать. Её меняете вы, а не агент
- Полезно как словарь: когда в статье или фреймворке пишут «memory», спросите, которая из четырёх

Эпизодическая память: лог того, что было

- Каждый диалог или прогон дописываем в журнал: вопрос, ответ, время, важные шаги
- Хранить дешево, читать целиком дорого. В промпт кладут не весь журнал, а последние записи или найденные по запросу
- Из эпизодов рождается семантическая память: модель перечитывает их и извлекает факты
- На первое время хватит одного JSON на запись в файл. Поиск по журналу появится с пятой лекции



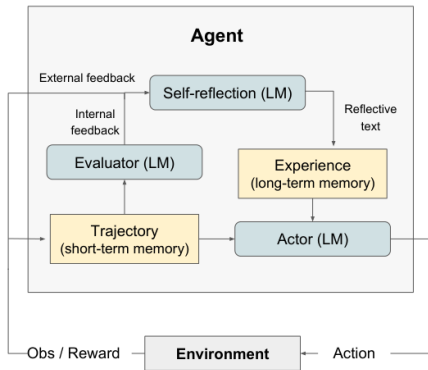
Memory stream у генеративных агентов: тот же журнал, из которого достают по свежести, важности и близости к запросу

Источник: Park et al., 2023

```
prompt = ("Извлеки из диалога факты о "  
         "пользователе, которые пригодятся "  
         "в будущих разговорах: имя, город, "  
         "предпочтения. Верни JSON-массив "  
         "коротких строк. Только НОВЫЕ.\n"  
         f"Уже известно: {facts}\n"  
         f"Диалог: {dialog}")  
  
new = json_from(model(prompt))  
facts = merge(facts, new) # с разрешением  
                           # противоречий
```

- Факты извлекает сама модель после ответа, отдельным дешёвым вызовом
- Строгий формат (JSON-массив), чтобы факты читала программа
- Просим только **новое** относительно известного, иначе память распухнет дублями
- В следующий запуск факты идут прямо в системный промпт: «пользователь Макар из Казани»

Память не только о пользователе: агент запоминает свои ошибки



Источник: Shinn et al., 2023. Reflexion

- Reflexion: после неудачной попытки отдельный вызов оценивает результат, а модель пишет себе **вывод словами**: что пошло не так и что делать иначе
- Вывод кладут в долгую память, и следующая попытка начинается уже с ним в контексте
- Никакого дообучения, только текст в памяти. На задачах по коду это дало 91 процент решённых против 80 у той же модели без рефлексии
- Механизм тот же, что извлечение фактов: модель перечитывает эпизод и оставляет короткую запись на будущее

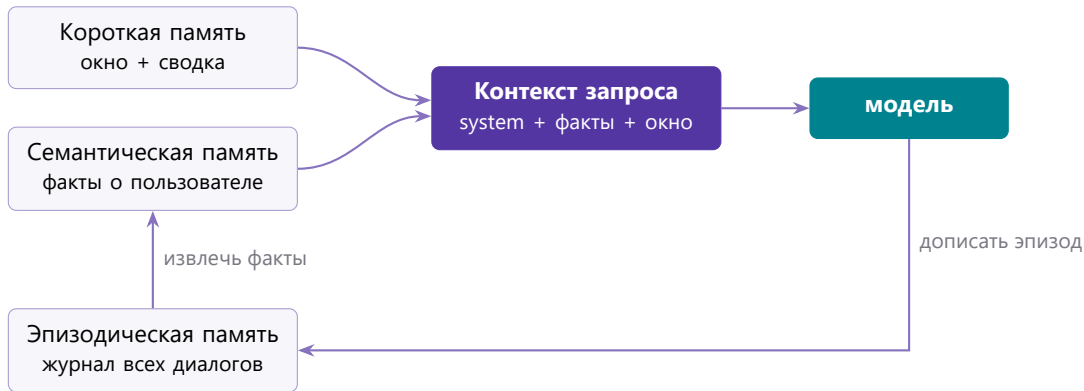
Противоречия

- «Живу в Казани» сегодня и «переехал в Москву» через месяц. Оба факта в памяти лгут вместе
- Новый факт должен уметь **перезаписать** старый, а не лечь рядом
- Простое правило: при слиянии сравнить с близкими фактами и заменить устаревший

Забывание

- Память без забывания это свалка, где важное тонет
- Что выкидывать: разовые мелочи, устаревшее, то, что не пригодилось ни разу
- Что беречь: устойчивые предпочтения, идентичность, повторяющиеся темы
- Забывание это не баг, а **политика**, которую вы задаёте

Рабочая архитектура памяти агента



В контекст идёт только короткая память и факты. Журнал и извлечение фактов работают в стороне, между запросами.

- Провайдер кэширует стабильное начало запроса и берёт за него меньше. Об этом была первая лекция
- Память легко ломает кэш: если пихать свежие факты и сводки **в начало**, префикс меняется каждый вызов и кэш не срабатывает
- Держите стабильное впереди: системный промпт, устойчивые факты. Переменное, свежую сводку и последние реплики, ближе к концу

Проверка

Смотрите usage: если `cache_read` на повторных вызовах ноль, значит память переписывает префикс. Переставьте порядок.

- Много задач **без состояния**: классификация, извлечение, разовый ответ. Тащить туда память значит платить впустую
- Долгая память это не галочка «сделать умнее», а ответ на вопрос «что именно между сессиями нужно помнить и зачем»
- Если не можете назвать факт, который стоит пережить перезапуск, память вам пока не нужна

Соблазн

«Давайте на всякий случай запоминать всё» это прямой путь к дорогой свалке, из которой модель потом достаёт мусор. Память добавляют под конкретную потребность.

- После каждого ответа отдельный дешёвый вызов извлекает факты о пользователе в JSON
- Факты лежат в файле, эпизоды дописываются в журнал строкой на диалог
- В следующий запуск факты и последние вопросы идут в системный промпт
- На вебинаре это выглядело так: первый запуск «меня зовут Макар, я из Казани», второй начинается с «Макар, мы уже обсуждали»

Честная оговорка

Извлечение фактов делает та же модель, и иногда ошибается: путает, дублирует, теряет. Поэтому память тоже надо **проверять на наборе**, как и всё остальное. Об этом на шестой неделе.

Готовые решения, если не хочется писать своё



Letta (бывший
MemGPT)



Mem0



Zep



LangGraph



Redis

- **Letta** и **Mem0**: память как сервис. Сами извлекают факты, разрешают противоречия, отдают нужное в промпт
- **Zep**: журнал диалогов плюс граф фактов с датами, удобно для противоречий во времени
- **LangGraph**: чекпоинты состояния и хранилище на уровне фреймворка, об этом на третьей неделе
- **Redis** или обычная база: где всё это лежит в проде. Файл с JSON хорош, пока пользователь один

Внутри у всех то же, что вы соберёте на семинаре руками: журнал, извлечение фактов, поиск по ним. Сначала понять механику, потом брать готовое.

- ❶ Тащить всю историю без окна и суммаризации: цена растёт квадратично, на длинном диалоге счёт улетает
- ❷ Складывать важное в середину длинного контекста и удивляться, что модель его не видит
- ❸ Резать скользящим окном вместе с системным промптом: агент забывает и правила, и задачу
- ❹ Запоминать всё подряд: память превращается в свалку, из которой достаётся мусор
- ❺ Класть свежие факты в начало промпта и убивать кэш
- ❻ Не разрешать противоречия: старый и новый факт лгут вместе
- ❼ Верить извлечённым фактам без проверки: модель ошибается и в извлечении

Руками

- Добавить агенту скользящее окно и суммаризацию по порогу
- Эпизодический журнал диалогов
- Извлечение фактов о пользователе в JSON с разрешением противоречий
- Проверить: помнит ли агент вас во втором запуске

Дальше: как вообще устроен поиск нужного по смыслу. Эмбединги и векторные базы.

Домашка

Дать агенту долговременную память поверх RAG-агента этой недели: журнал плюс факты. К следующей неделе это соединится с поиском по документам.

- ❶ У модели нет памяти. Память агента это то, что ваш код кладёт в контекст на каждом шаге, и платите вы за это каждый вызов
- ❷ Контекстное окно это бюджет, а не склад. Кладём минимум, которого хватит для следующего шага
- ❸ Даже когда всё влезает, длинный контекст ненадёжен: важное, зарытое в середину, теряется
- ❹ Короткая память живёт окном и суммаризацией. Долгая делится на эпизодическую и семантическую
- ❺ Память добавляют под конкретную потребность, с забыванием и разрешением противоречий, и тоже проверяют

Вопросы?

следующая лекция: эмбединги и векторные базы, как искать нужное по смыслу, а не по словам